LABORATORIO 2026

Formularios

Cómo recolectar datos del usuario en una página web



Objetivos de la clase

Al finalizar el encuentro, el estudiante va a poder:

1. **Entender la arquitectura cliente-servidor y qué es un formulario HTML.**
2. **Conocer los atributos principales de `<form>` (`action`, `method`) y los tipos de `<input>` más usados.**
3. **Asociar correctamente `<label>` con su control usando `for`/`id`.**
4. **Usar `<select>`/`<option>`, `<textarea>` y `<button>`.**
5. **Aplicar validación nativa básica (`required`, `placeholder`, `min`/`max`) y buenas prácticas de accesibilidad.**

Requisitos para esta clase

- **Tu *ficha.html* de la clase anterior**

  Abierto en VSC, con Live Server activo

- **Etiquetas semánticas *(header/nav/main/footer)* ya aplicadas**

01 SECCIÓN 01

Cliente y servidor

Cómo se comunican el navegador y el servidor: solicitud, respuesta y métodos HTTP

Arquitectura cliente-servidor

Modelo en el que un **cliente** pide recursos (páginas, imágenes, datos) y un **servidor** los procesa y le responde.

LOS DOS ROLES

Cliente: el navegador (Chrome, Firefox, Edge)
Servidor: la computadora remota donde vive el sitio

> **La comunicación es siempre solicitud → respuesta: el cliente pregunta y el servidor contesta. El idioma que usan para hablar se llama HTTP.**



Ejemplo: al entrar a *google.com*, tu navegador le pide la página al servidor de Google, y este le devuelve el HTML que ves en pantalla.

El ciclo solicitud → respuesta

Lo que pasa cada vez que **abrís una página** o **enviás un formulario**:

1. **El cliente arma una solicitud**
   Escribís una URL o tocás un botón de *Enviar*: el navegador **prepara una solicitud HTTP**.

2. **La solicitud viaja al servidor**
   Indica un método (`GET`, `POST`…) **y el recurso** que se pide (por ejemplo `/contacto`).

3. **El servidor la procesa**
   Busca el archivo pedido, guarda los datos recibidos o consulta una base de datos.

4. **El servidor devuelve una respuesta**
   Un código de estado (`200` OK, `404` No encontrado) y el contenido, por ejemplo el HTML de la página.

Métodos HTTP

Un método HTTP es el verbo que indica qué **acción** se quiere realizar sobre un recurso del servidor.

MÉTODOS ESTÁNDAR

`GET` · `POST` · `PUT` · `PATCH` · `DELETE`

> **En HTML, el atributo `method` de `<form>` solo admite `GET` y `POST` de forma nativa — son los que vamos a usar durante toda esta clase.**

En la web con formularios HTML, la gran mayoría del tráfico es `GET` y `POST`; el resto (`PUT`, `PATCH`, `DELETE`…) aparece más adelante, en APIs.

GET vs *POST*

GET

```
<form action="/buscar" method="get">
```

GET /buscar?q=laboratorio

Los datos van en la URL (querystring): visibles, con límite de tamaño, se pueden compartir como link.

POST

```
<form action="/contacto" method="post">
```

POST /contacto

nombre=Ana&correo=ana@mail.com

Los datos van en el cuerpo de la solicitud: no se ven en la URL, sin límite práctico de tamaño.

CUÁNDO USAR CADA UNO

GET para **consultar** o **buscar**.

POST para **enviar**, **crear** o **modificar** datos (login, formularios de contacto, datos sensibles).

Nota: un formulario HTML no manda JSON. Con *POST*, el navegador arma el cuerpo en un formato llamado *application/x-www-form-urlencoded*: pares *nombre=valor* unidos con *&*, igual que en la URL de *GET*. Algunos caracteres se codifican (el *@* viaja como *%40*).

02 SECCIÓN 02

Formularios

El elemento <form>, <input>, <label>, <select>, <textarea>, <button> y validación nativa

<form> — el contenedor del formulario

Elemento que agrupa los controles de **entrada de datos** y define **a dónde y cómo se envían**.
Atributos:

action: a dónde se envían los datos.
method: método a utilizar (***GET*** o ***POST***). Por defecto es ***GET*** si no se especifica.

EJEMPLO

```
<form action="/contacto" method="post">
  <!-- inputs van acá -->
</form>
```

Inputs de texto

Tipos de *<input>* para texto simple: *text*, *email* y *password*.

EJEMPLO

```html
<input type="text" name="nombre">
<input type="email" name="correo">
<input type="password" name="clave">
```



ERROR COMÚN

Usar *type="text"* para todo, se pierde el teclado adecuado (@ en mobile para email) y la validación nativa del navegador.

Nota: el atributo *name* es obligatorio, es la **clave** con la que viaja el dato al servidor.

Inputs numéricos y de fecha

Tipo *number* (con *min*/*max*) y tipo *date* para valores numéricos y fechas.

EJEMPLO

```
<input type="number" name="edad" min="1" max="99">
<input type="date" name="nacimiento">
```



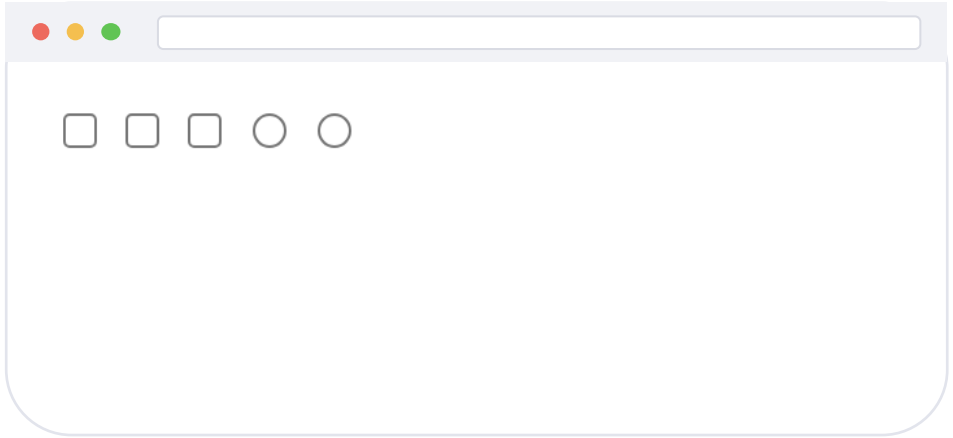
Inputs de selección

Tipo ***checkbox*** (opción **independiente**) y tipo ***radio*** (opciones **excluyentes**; se agrupan con el mismo ***name***).

EJEMPLO

```
<input type="checkbox" name="acepto" id="acepto">
<input type="checkbox" name="novedades" id="novedades">
<input type="checkbox" name="terminos" id="terminos">

<input type="radio" name="contacto" value="email">
<input type="radio" name="contacto" value="telefono">
```



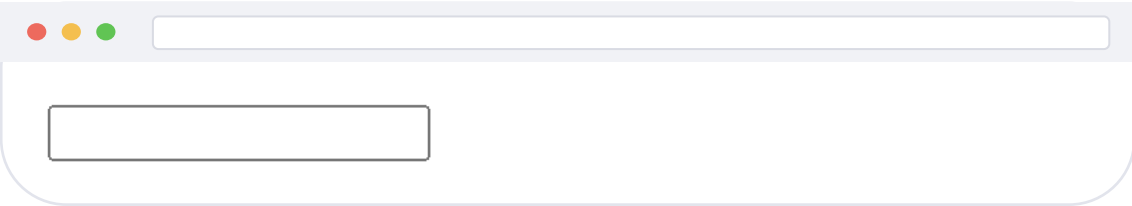
<label> y accesibilidad

Asocia un texto descriptivo a un control mediante `for` (en el `<label>`) e `id` (en el input). Por qué importa: click en el texto activa el control; los lectores de pantalla anuncian el campo.

COMPARACIÓN

✕ Sin label

```
<input type="email" name="correo">
```



✓ Con label

```
<label for="correo">Email</label>
<input type="email" id="correo" name="correo">
```



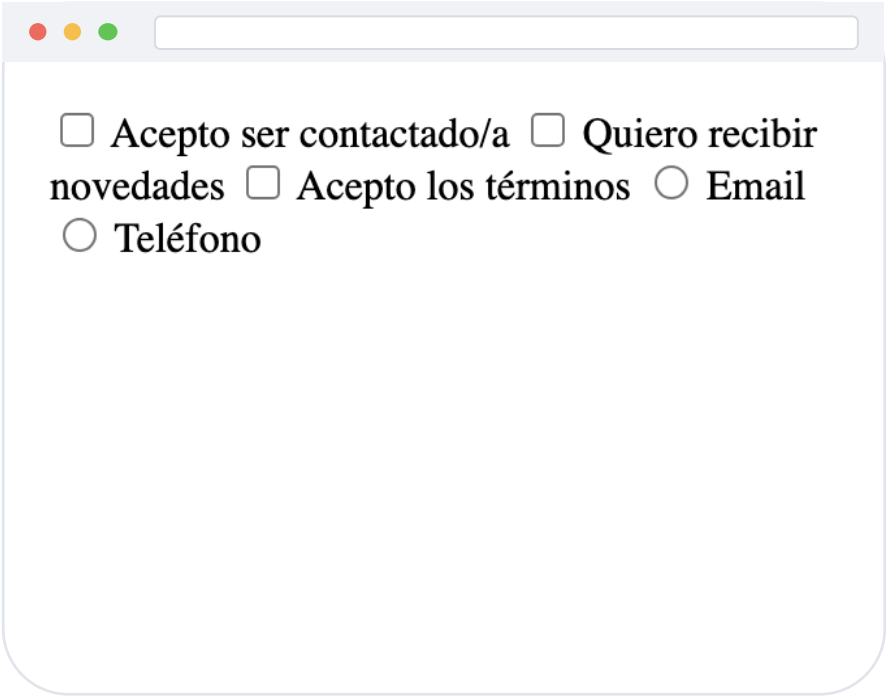
Inputs de selección con *<label>*

El mismo ejemplo, con un *<label>* por opción: ahora se lee **qué es cada una**, y hacer click en el texto marca el control.

EJEMPLO

```html
<input type="checkbox" name="acepto" id="acepto">
<label for="acepto">Acepto ser contactado/a</label>
<input type="checkbox" name="novedades" id="novedades">
<label for="novedades">Quiero recibir novedades</label>
<input type="checkbox" name="terminos" id="terminos">
<label for="terminos">Acepto los términos</label>

<input type="radio" name="contacto" value="email" id="email">
<label for="email">Email</label>
<input type="radio" name="contacto" value="telefono" id="telefono">
<label for="telefono">Teléfono</label>
```

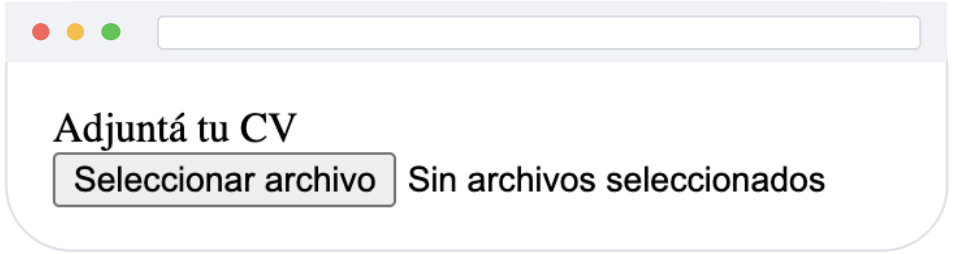


Input de archivo

Tipo *`file`*: muestra un botón que abre el selector de archivos de la computadora para adjuntar uno.

EJEMPLO

```html
<label for="cv">Adjuntá tu CV</label>
<input type="file" id="cv" name="cv">
```



Nota: para que el archivo viaje al servidor, el *`<form>`* tiene que usar *`method="post"`* y *`enctype="multipart/form-data"`*.

<select> y <option>

Lista desplegable de opciones: `<select>` agrupa varios `<option>`.

EJEMPLO

```html
<label for="asunto">Asunto</label>
<select name="asunto" id="asunto">
  <option value="consulta">Consulta</option>
  <option value="soporte">Soporte</option>
</select>
```



<textarea>

Campo de texto de varias líneas, pensado para mensajes largos.

EJEMPLO

```html
<label for="mensaje">Mensaje</label>
<textarea name="mensaje" id="mensaje" rows="4"></textarea>
```

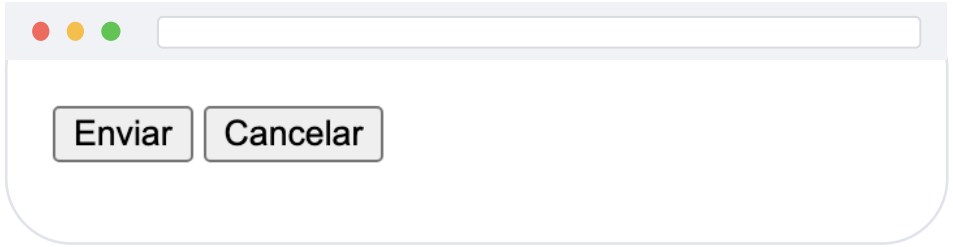


<button> y el envío del formulario

`<button type="submit">` envía el formulario
`<button type="button">` no hace nada por sí solo (se usa con JavaScript);

EJEMPLO

```html
<button type="submit">Enviar</button>
<button type="button">Cancelar</button>
```



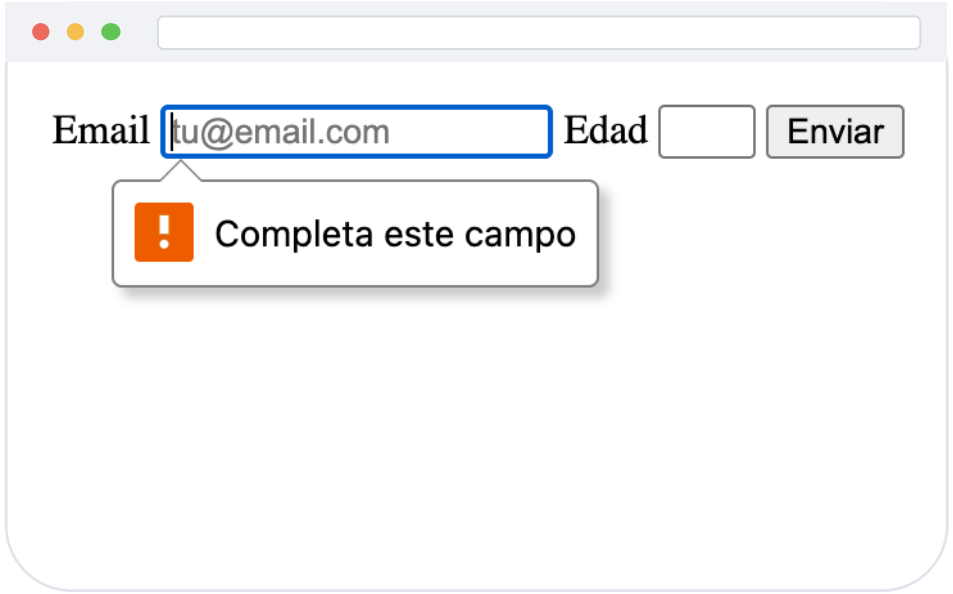
Un `<button>` sin `type` dentro de un `<form>` actúa como `submit` por defecto.

Validación nativa

Atributos que el navegador valida solo, sin JavaScript: ***required***, ***placeholder*** y ***min***/***max***.

EJEMPLO

```html
<form>
  <label for="correo">Email</label>
  <input type="email" id="correo" name="correo"
         required placeholder="tu@email.com">
  <label for="edad">Edad</label>
  <input type="number" id="edad" name="edad"
         min="18" max="99" required>
  <button type="submit">Enviar</button>
</form>
```



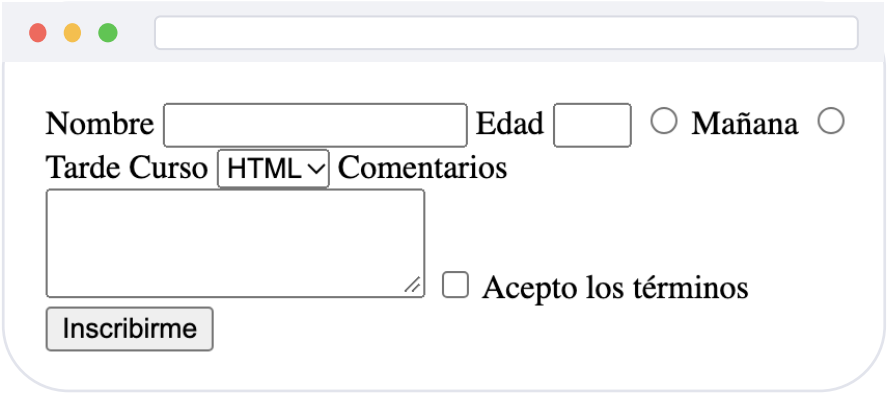
ERROR COMÚN

Confiar solo en la validación del navegador. También hay que validar en el servidor (tema para más adelante), porque la validación del cliente se puede saltear.

Un formulario completo

Un ejemplo que combina lo que vimos hoy: *<form>*, *<input>*, *<label>*, *<select>*, *<textarea>* y *<button>*.

```html
<form action="/inscripcion" method="post">
  <label for="nombre">Nombre</label>
  <input type="text" id="nombre" name="nombre" required>
  <label for="edad">Edad</label>
  <input type="number" id="edad" name="edad" min="16" max="99">
  <input type="radio" name="turno" value="manana"> Mañana
  <input type="radio" name="turno" value="tarde"> Tarde
  <label for="curso">Curso</label>
  <select id="curso" name="curso">
    <option value="html">HTML</option>
    <option value="css">CSS</option>
  </select>
  <label for="comentarios">Comentarios</label>
  <textarea id="comentarios" name="comentarios" rows="3"></textarea>
  <input type="checkbox" id="terminos" name="terminos" required> Acepto los términos
  <button type="submit">Inscribirme</button>
</form>
```



Repaso: Verdadero o Falso

Para cada afirmación, ¿verdadero o falso?

1. El atributo *name* de un input es opcional. **V** **F**
2. *for* en el label debe coincidir con el *id* del input. **V** **F**
3. Con *GET* los datos viajan en la URL, y con *POST* en el cuerpo de la solicitud. **V** **F**
4. Un *<button>* sin *type* dentro de un form actúa como submit. **V** **F**
5. *required* necesita JavaScript para validar en el navegador. **V** **F**

Repaso: Verdadero o Falso — Respuestas

La opción correcta queda marcada en dorado.

1	El atributo *name* de un input es opcional.	V	**F**
2	*for* en el label debe coincidir con el *id* del input.	**V**	F
3	Con *GET* los datos viajan en la URL, y con *POST* en el cuerpo de la solicitud.	**V**	F
4	Un *<button>* sin *type* dentro de un form actúa como submit.	**V**	F
5	*required* necesita JavaScript para validar en el navegador.	V	**F**

03 SECCIÓN 03

Práctica

Formulario de contacto sobre tu ficha personal



Ejercicio 1: barra de navegación

1. Crear *contacto.html* en la misma carpeta que *ficha.html*, con la misma estructura: *<header>*, *<nav>*, *<main>* y *<footer>*.

2. En el *<nav>* de las dos páginas, dejar solo dos enlaces: INICIO y CONTACTO. (Repetido por ahora)

3. INICIO apunta a *ficha.html* y CONTACTO a *contacto.html*, con el atributo *href*.

4. Probar con Live Server: hacer click en cada enlace e ir y volver entre las dos páginas.

```html
<nav>
  <a href="ficha.html">INICIO</a>
  <a href="contacto.html">CONTACTO</a>
</nav>
```

Ejercicio 2: el formulario en contacto.html

1. Dentro del *<main>* de *contacto.html*, agregar una *<section>* "Contactame" con un *<h2>*.

2. Campos: nombre, email , asunto (*select* con 3 opciones), mensaje (*textarea*).

3. Un *checkbox* "Acepto ser contactado/a".

4. Cada input con su *<label>* asociado por *for*/*id*.

5. Botón de envío *<button type="submit">*.

6. Probar la validación nativa: enviar vacío, probar un email inválido.

```html
<section>
  <h2>Contactame</h2>
  <form action="#" method="post">
    <label for="nombre">Nombre</label>
    <input type="text" id="nombre" name="nombre" required>
    <label for="correo">Email</label>
    <input type="email" id="correo" name="correo" required>
    <label for="asunto">Asunto</label>
    <select id="asunto" name="asunto">
      <option value="consulta">Consulta</option>
      <option value="soporte">Soporte</option>
      <option value="otro">Otro</option>
    </select>
    <label for="mensaje">Mensaje</label>
    <textarea id="mensaje" name="mensaje" rows="4"></textarea>
    <label for="acepto">
      <input type="checkbox" id="acepto" name="acepto">
      Acepto ser contactado/a
    </label>
    <button type="submit">Enviar</button>
  </form>
</section>
```



¿Preguntas?